



UNIVERSIDADE DE SÃO PAULO
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação

Performance Evaluation of WebAssembly vs. JavaScript on
Linux-based Mobile Architectures: Impact of Input Size on
Numeric and String Operations

Murilo Fonseca de Matos



São Carlos – SP

Performance Evaluation of WebAssembly vs. JavaScript on Linux-based Mobile Architectures: Impact of Input Size on Numeric and String Operations

Murilo Fonseca de Matos

***Orientador:* Profa. Dra. Sarita Mazzini Bruschi**

Monografia final de conclusão de curso apresentada ao Instituto de Ciências Matemáticas e de Computação – ICMC-USP, como requisito parcial para obtenção do título de Bacharel em Computação. *Área de Concentração:* Sistemas distribuídos e programação concorrente

USP – São Carlos
Junho de 2026

Matos, Murilo Fonseca de

Performance Evaluation of WebAssembly vs. JavaScript
on Linux-based Mobile Architectures: Impact of Input
Size on Numeric and String Operations / Murilo Fonseca
de Matos. - São Carlos - SP, 2026.

54 p.; 29,7 cm.

Orientador: Sarita Mazzini Bruschi.

Monografia (Graduação) - Instituto de Ciências
Matemáticas e de Computação (ICMC/USP), São Carlos -
SP, 2026.

1. WebAssembly. 2. JavaScript. 3. Mobile
Performance. 4. V8 Engine. 5. Systems Architecture.
6. Benchmarking. I. Bruschi, Sarita Mazzini. II.
Instituto de Ciências Matemáticas e de Computação
(ICMC/USP). III. Título.

Dedico este trabalho aos meus pais, que ensinaram-me a amar o conhecimento

AGRADECIMENTOS

Dedico os meus agradecimentos a todos que me apoiaram nesta jornada, em especial à minha família, aos meus colegas de laboratório e à Professora Doutora Sarita Mazzini

*“As invenções são, sobretudo,
o resultado de um trabalho de teimoso.”
(Santos Dumont)*

RESUMO

MATOS, M. F.. **Performance Evaluation of WebAssembly vs. JavaScript on Linux-based Mobile Architectures: Impact of Input Size on Numeric and String Operations**. 2026. 54 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

Hoje, os smartphones executam softwares exigentes, como aplicativos de edição de fotos em tempo real, motores de jogos para dispositivos móveis e modelos de aprendizado de máquina no próprio dispositivo. Isso transfere uma carga de trabalho pesada para os dispositivos móveis, forçando os engenheiros de software a escolher entre duas ferramentas completamente diferentes: o JavaScript tradicional, amigável para programadores, ou o sistema WebAssembly, mais recente e de alta velocidade. Para descobrir qual ferramenta funciona melhor quando o hardware de um smartphone é levado ao seu limite absoluto, esta tese compara diretamente o mecanismo V8 do JavaScript com o WebAssembly (usando Wasmtime) em um processador móvel ARM64 padrão. Usando um script automatizado dentro de um ambiente sandbox seguro, submetemos ambos os sistemas a milhões de loops matemáticos e testes intensivos de mesclagem de texto.

Os resultados mostraram que a melhor ferramenta depende inteiramente do tipo de trabalho que está sendo feito. Para cálculos matemáticos pesados, o WebAssembly superou completamente o JavaScript usando uma quantidade ínfima e totalmente uniforme de memória. Isso prova que o código pré-compilado do WebAssembly é muito mais eficiente do que o sistema do JavaScript, que precisa traduzir e otimizar o código enquanto ele está em execução. No entanto, ao lidar com manipulação intensa de texto, descobrimos uma grande desvantagem. O WebAssembly permaneceu incrivelmente rápido, mas seu uso de memória explodiu, ultrapassando 100.000 KB, porque ele precisa continuamente solicitar novos blocos de memória do sistema operacional do smartphone. O JavaScript, por outro lado, usou seu coletor de lixo automático para limpar perfeitamente os dados de texto antigos, mantendo sua memória completamente estável — até atingir um limite máximo no maior tamanho de teste e travar com um erro fatal. Em última análise, esta comparação mostra que o WebAssembly pode superar os limites de desempenho e as falhas que interrompem os aplicativos JavaScript tradicionais, mas requer muito cuidado manual extra para gerenciar a memória. Portanto, para aplicativos móveis, nenhum dos sistemas é universalmente melhor; os engenheiros devem equilibrar cuidadosamente se seu aplicativo precisa de controle de memória sólido como uma rocha ou pura velocidade de execução.

Palavras-chave: WebAssembly, JavaScript, Mobile Performance, V8 Engine, Systems Architecture, Benchmarking.

ABSTRACT

MATOS, M. F.. **Performance Evaluation of WebAssembly vs. JavaScript on Linux-based Mobile Architectures: Impact of Input Size on Numeric and String Operations**. 2026. 54 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

Today, smartphones run demanding software like real-time photo editing apps, mobile gaming engines, and on-device machine learning models. This shifts a heavy workload onto mobile devices, forcing software engineers to choose between two completely different tools: traditional, programmer-friendly JavaScript or the newer, high-speed WebAssembly system. To find out which tool works best when a smartphone's hardware is pushed to its absolute limits, this thesis directly compares JavaScript's V8 engine against WebAssembly (using Wasmtime) on a standard ARM64 mobile processor. Using an automated script inside a safe sandbox environment, we pushed both systems through millions of mathematical loops and intensive text-merging tests. The results showed that the best tool depends entirely on the type of work being done. For heavy math calculations, WebAssembly completely crushed JavaScript using a tiny, completely flat amount of memory. This proves that WebAssembly's pre-compiled code is much more efficient than JavaScript's system, which has to translate and optimize code while it is running. However, when handling heavy text manipulation, we discovered a major trade-off. WebAssembly stayed incredibly fast, but its memory usage exploded past 100,000 KB because it continuously has to request new memory chunks from the smartphone's operating system. JavaScript, on the other hand, used its automatic Garbage Collector to clean up old text data perfectly, keeping its memory completely flat—until it hit a hard ceiling at the highest test size and crashed with a fatal error. Ultimately, this comparison shows that WebAssembly can break through the performance limits and crashes that stop traditional JavaScript apps, but it requires a lot of extra manual care to manage memory. Therefore, for mobile apps, neither system is universally better; engineers must carefully balance whether their app needs rock-solid memory control or pure execution speed.

Key-words: WebAssembly, JavaScript, Mobile Performance, V8 Engine, Systems Architecture, Benchmarking.

LISTA DE ILUSTRAÇÕES

Figura 1 – Peak Memory Consumption Comparison for Numeric Workloads	35
Figura 2 – Execution Latency Scaling Trend for Numeric Workloads	36
Figura 3 – Peak Memory Consumption Comparison for string Workloads	36
Figura 4 – Execution Latency Scaling Trend for string Workloads	37

LISTA DE TABELAS

Tabela 1 – Experimental Input Sizes	29
Tabela 2 – Performance Speedup Factors (S)	39

LISTA DE CÓDIGOS-FONTE

1	Equivalent Numerical Benchmark in JavaScript (numeric_test.js)	47
2	Numerical Benchmark Implementation in C (numeric_test.c)	47
3	string Benchmark Implementation in C (string_test.c)	48
4	string Benchmark Implementation in JS (string_test.js)	49
5	Automated Data Collection Orchestration Harness (benchmark_sh)	49
6	python script chart	50

SUMÁRIO

1	INTRODUCTION	21
1.1	Motivation and Contextualization	21
1.2	Objectives	22
1.3	Organization	22
2	THEORETICAL BACKGROUND	23
2.1	Virtualization and Dynamic Execution Environments	23
2.1.1	<i>The JavaScript Execution Model and the V8 Engine</i>	23
2.1.2	<i>WebAssembly and Standalone Runtimes</i>	24
2.2	Operating System Resource Telemetry	25
2.2.1	<i>Virtual Memory and Physical Space Allocation</i>	25
2.2.2	<i>Resident Set Size (RSS) and Runtime Resource Telemetry</i>	26
3	RELATED WORK	27
3.0.1	<i>Virtual Machine Telemetry and JIT Volatility</i>	27
3.0.2	<i>WebAssembly and WASI in Standalone Environments</i>	27
3.1	Comparative Runtime Benchmarking and Performance Abstractions	28
4	METHODOLOGY	29
4.1	Configuration and Compilation	29
4.2	Deployment Instrumentation and Telemetry Capture	30
4.3	Data collection	31
5	DEVELOPMENT	33
5.1	The problem	33
5.2	Realized activities	33
5.3	Results	34
5.3.1	<i>Numeric trials results</i>	34
5.3.2	<i>String trials results</i>	36
5.3.3	<i>S factor table</i>	38
5.4	Difficulties and Limitations	39
5.4.1	<i>Methodological and Architectural Limitations</i>	39
5.4.2	<i>System Administration and Linux Environment Traps</i>	39
5.4.3	<i>Runtime Memory Constraints</i>	39

6	CONCLUSION	41
7	CITATIONS AND REFERENCES	43
	REFERENCES	45
APÊNDICE A	SCRIPTS	47

INTRODUCTION

1.1 Motivation and Contextualization

Unlike desktop environments, mobile hardware is inherently constrained by limited thermal envelopes, varying power efficiency, and complex hardware abstraction layers, making it easy to exceed a device’s computational capacity. Despite not being locally installed in the traditional sense, web applications constantly utilize these limited hardware resources. Consequently, it is vital to investigate the underlying technologies that drive the web within mobile contexts.

JavaScript is undeniably the most prevalent language for constructing web-based applications, powering everything from standard websites to complex tools like Gmail and Google Maps (FLANAGAN, 2020). While JavaScript is a versatile and powerful tool, it possesses inherent architectural bottlenecks—specifically regarding execution performance. As an interpreted and Just-In-Time (JIT) compiled language, the browser must parse, optimize, and compile code during runtime (YAN *et al.*, 2021). This process introduces significant overhead, which is further exacerbated in a mobile context where CPU and battery resources are at a premium.

To address these limitations and meet the increasing demands of web users, WebAssembly (Wasm) was developed. WebAssembly is a binary instruction format designed to execute at near-native speeds. As a low-level bytecode—often compiled from high-level languages like C, C++, or Rust—Wasm enables web engines to execute computationally intensive tasks far more efficiently than traditional JavaScript (YAN *et al.*, 2021). Its emergence has paved the way for running heavy applications, such as 3D games and high-performance video processing, directly in the browser, offering significant performance benefits for mobile devices.

While both JavaScript and WebAssembly are powerful, they serve different purposes, making performance comparisons inevitable. The optimal choice often depends on the specific use case. For mobile environments, it is crucial to understand how these technologies behave on Linux-based architectures—such as Android—which dominate the global mobile market (Statcounter Global Stats, 2026). This research focuses on evaluating these behaviors through a “Performance Evaluation of WebAssembly vs. JavaScript on Linux-based Mobile Architectures”, specifically analyzing the impact of input size on numeric and string operations.

1.2 Objectives

The general objective of this work analyse the performance of Javascript and WebAssembly on Linux-based Mobile Architectures. The study seeks to investigate how different types of workload — specifically intensive numerical operations and string manipulation behave under variations in input size.

As specific objectives, the following stand out:

1. Implement and evaluate a method to compare WebAssembly and Javascript for numeric and string operations.
2. Identify performance's bottlenecks and evaluate the scalability of both approaches.
3. Statistical comparison between of the data obtained.

1.3 Organization

Chapter 2 describes the experimental methodology adopted, including the models implementation used. Chapter 3 presents the activities carried out and discusses the results obtained from different analytical perspectives. Finally, Chapter 4 summarizes the contributions of the work, its limitations, and possible directions for future research.

THEORETICAL BACKGROUND

2.1 Virtualization and Dynamic Execution Environments

2.1.1 *The JavaScript Execution Model and the V8 Engine*

JavaScript is architected as a high-level, dynamically typed language, meaning that type checking occurs at runtime rather than during a compilation phase, and variables can hold data of any type fluidly. While this abstraction provides significant development flexibility, it introduces severe performance penalties for executing hardware directly, as the underlying processor requires rigid types and fixed memory layouts to execute instructions efficiently. Empirical studies show that this dynamic behavior forces the runtime process to constantly evaluate object properties, altering expected optimization paths (RICHARDS *et al.*, 2010). To bridge this gap, modern execution environments do not simply interpret the source code; they rely on sophisticated runtime engines.

In the context of the Node.js runtime environment, this execution layer is governed by Google's V8 engine. Rather than relying on traditional ahead-of-time compilation, V8 employs a hybrid execution strategy known as Just-In-Time (JIT) compilation. The JIT pipeline is designed to eliminate translation overhead by compiling JavaScript code directly into native machine code during execution, utilizing a multi-tiered architecture that balances startup latency against peak optimization.

The process begins within V8's interpreter, named Ignition. Upon receiving the source code, Ignition compiles it into an intermediate representation called bytecode and begins immediate execution. Because bytecode generation is extremely fast, this tier minimizes the initial startup delay, often characterized as "warm-up latency" in virtual machine micro-benchmarks (BARRETT *et al.*, 2017).

As Ignition executes the bytecode, a dedicated profiling thread continuously monitors the application's telemetry. The profiler tracks execution frequency and data structures, identifying functions or specific loops that run repeatedly—classifying them as "hot code." Crucially, the profiler records the precise types passing through these execution paths, establishing a baseline of speculative type assumptions.

Once a code segment passes a threshold of execution frequency, V8 passes the bytecode and the collected profiling metadata to its high-tier optimizing compiler, TurboFan. TurboFan

leverages the speculative types to perform aggressive, low-level optimization passes, such as function inlining, loop invariant code motion, and register allocation. This tier maps frequently accessed numeric variables directly onto the physical CPU hardware registers to achieve near-native execution performance.

The output of TurboFan is highly optimized, native machine code tailored to the host processor architecture. If the speculative type assumptions remain true, the engine executes this fast path at speeds approaching statically compiled native languages.

However, because JavaScript is dynamically typed, these assumptions can be violated at any moment. If an optimized loop encounters an unexpected runtime type mutation, the hardware instructions become invalid. When this occurs, the engine triggers a Deoptimization (Bailout) cycle. The optimized machine code is discarded, the execution state is safely reconstructed, and the engine falls back to the Ignition interpreter. This continuous loop of profiling, optimization, and potential deoptimization introduces significant performance jitter and statistical variance in execution time under high-iteration workloads (BARRETT *et al.*, 2017).

2.1.2 *WebAssembly and Standalone Runtimes*

In contrast to the dynamic interpretation and speculative compilation model of traditional web languages, WebAssembly (Wasm) provides a low-level, size- and time-efficient binary instruction format designed as a portable compilation target for high-performance languages such as C, C++, and Rust (ZHANG *et al.*, 2024). Structurally, WebAssembly operates as a stack-based virtual machine designed to run sandboxed code at near-native speed. Unlike JavaScript, Wasm enforces strict static typing, meaning that the types of all variables, function signatures, and memory layouts are fixed and validated during the pre-execution compilation phase rather than discovered incrementally during runtime execution (RAMESH *et al.*, 2023).

While WebAssembly was originally conceived for execution within web browsers alongside JavaScript engines, the development of the WebAssembly System Interface (WASI) expanded its capabilities to standalone, server-side, and embedded execution environments (RAMESH *et al.*, 2023). In a standalone environment, the execution layer is managed by specialized, lightweight runtimes rather than broad browser suites (ZHANG *et al.*, 2024). In this study, Google's V8 engine is contrasted against Wasmtime, a high-performance, standalone WebAssembly runtime built on the Cranelift compiler backend (ENGELKE; SCHWARZ, 2024).

The fundamental architectural distinction between a JIT-driven engine like V8 and a standalone runtime like Wasmtime lies in the compilation timeline and the complete elimination of runtime telemetry monitoring. Wasmtime primarily leverages Ahead-of-Time (AOT) compilation. In this execution model, the low-level Wasm binary bytecode is fully compiled into native machine code before the application begins its execution phase.

Because WebAssembly binaries already contain strict type definitions, Wasmtime does

not need to warm up, interpret bytecode initially, or profile data types dynamically at runtime. This architectural layout completely bypasses the runtime profiling tax—the CPU cycle overhead that JIT engines must pay to observe code behavior, manage heuristic thresholds, and detect type mutations.

2.2 Operating System Resource Telemetry

2.2.1 *Virtual Memory and Physical Space Allocation*

To manage hardware resources efficiently and provide security isolation between executing processes, modern operating systems do not allow application runtimes to access physical random-access memory (RAM) directly. Instead, the operating system kernel introduces a layer of abstraction known as the Virtual Memory System ([SILBERSCHATZ; GALVIN; GAGNE, 2018](#)). Under this architecture, every executing process—such as a Node.js instance or a Wasm-time host process—is allocated its own continuous, private range of addresses called a Virtual Address Space.

The abstraction of virtual memory relies on a fundamental architectural mechanism known as Paging. The operating system divides both virtual and physical memory into fixed-size blocks: Virtual Pages which have uniform segments that make up a process’s virtual address space (typically 4 KB in standard architectures) and physical Page Frames which have the actual physical slots available on the hardware RAM sticks.

The hardware-level translation between these two domains is governed by the CPU’s Memory Management Unit (MMU) via an internal kernel data structure called the Page Table ([SILBERSCHATZ; GALVIN; GAGNE, 2018](#)). When a runtime requests memory from the system (for instance, via a standard `malloc()` call or an internal engine expansion), the operating system initially allocates only virtual addresses. No physical RAM hardware is immediately consumed.

The physical allocation of hardware RAM follows a lazy-loading strategy known as Demand Paging ([TANENBAUM; BOS, 2015](#)). When the process attempts to read or write to an allocated virtual page for the first time, the MMU intercepts the operation. Because the virtual page does not yet point to a physical location, the hardware triggers a synchronous interrupt called a Page Fault.

Upon intercepting a page fault, the operating system kernel temporarily pauses the execution thread, locates an empty physical page frame on the physical RAM, maps it to the requesting virtual address within the page table, and resumes execution ([TANENBAUM; BOS, 2015](#)).

Understanding this distinction between lazy virtual reservation and active physical mapping is essential for resource telemetry benchmarking. If an execution engine aggressively

allocates virtual memory but rarely populates it with data, its virtual footprint will appear artificially massive, while its actual burden on the physical hardware infrastructure remains minimal.

2.2.2 Resident Set Size (RSS) and Runtime Resource Telemetry

While virtual memory size represents the total address space reserved by a process, it does not reflect actual hardware utilization. In empirical performance evaluation and resource telemetry, the critical metric for measuring real memory consumption is the Resident Set Size (RSS) (MARICQ *et al.*, 2018). Formally, RSS is defined as the exact portion of a process's allocated memory that currently resides within the host device's physical random-access memory (RAM) channels.

RELATED WORK

The optimization and resource evaluation of managed runtime environments constitute a well-established domain in systems research. Existing literature closely related to this study can be systematically classified into three primary axes: virtual machine performance characteristics, server-side WebAssembly isolation paradigms, and empirical runtime benchmarking.

3.0.1 *Virtual Machine Telemetry and JIT Volatility*

The behavior of Just-In-Time (JIT) optimizing compilers under high-iteration, compute-bound workloads has been heavily examined. A foundational study by (RICHARDS *et al.*, 2010) performed an extensive empirical evaluation of JavaScript applications, demonstrating that the highly dynamic nature of the language forces execution engines to maintain intensive runtime type-checking guards, which frequently disrupt compiler optimizations.

Furthermore, the predictability of JIT-driven virtual machines was problematized by (BARRETT *et al.*, 2017). Their work illustrated that multi-tiered JIT environments—such as Google’s V8 engine—exhibit significant performance volatility and "warm-up jitter." They proved that execution paths optimized by components like TurboFan can unexpectedly deoptimize due to subtle telemetry variations, leading to a lack of determinism in execution times. This body of work underscores the inherent hardware resource costs and performance unpredictability that this study observes when executing long-running loops in Node.js.

3.0.2 *WebAssembly and WASI in Standalone Environments*

With the standardization of the WebAssembly System Interface (WASI), a growing body of literature has explored the migration of WebAssembly from browser environments to standalone, server-side, and edge computing deployments. A comprehensive architectural survey by (ZHANG *et al.*, 2024) evaluated contemporary standalone WebAssembly runtimes, highlighting that ahead-of-time (AOT) compilation strategies offer highly predictable execution boundaries and minimized memory footprints compared to traditional containerization layers.

Similarly, (RAMESH *et al.*, 2023) investigated the system call interface abstractions of WASI, noting that executing sandboxed binaries via direct system mapping minimizes host kernel transition overheads. These studies provide the justification for utilizing Wasmtime as a baseline, confirming its capability to operate as a low-latency alternative to high-level languages.

3.1 Comparative Runtime Benchmarking and Performance Abstractions

Crucial baseline insights were established by (YAN *et al.*, 2021), who conducted a large-scale systematic study analyzing the performance anomalies of WebAssembly applications in relation to traditional JavaScript execution. Their findings highlighted a critical counter-intuitive reality: because frontend WebAssembly compilers are fundamentally built on top of generic compiler infrastructures (such as LLVM), their internal optimizations are not natively tailored for WebAssembly bytecode targets, occasionally rendering traditional compiler optimization passes ineffective.

Furthermore, (YAN *et al.*, 2021) observed that while JIT optimization passes dramatically accelerate dynamic languages by tracing code telemetry, they yield no substantial performance gains for WebAssembly modules, which inherently rely on fixed-type layouts. Strikingly, their empirical data also revealed that WebAssembly binaries frequently consume significantly more memory than their equivalent JavaScript counterparts during specific allocation tasks.

While the foundational study by (YAN *et al.*, 2021) successfully exposes macro-level execution and memory trends across diverse compiler suites, their evaluation primarily focused on browser-based engines and general application frameworks. This represents the precise academic gap filled by this study: this work shifts the environment entirely to server-side, standalone runtimes, providing a fine-grained, empirical analysis contrasting the low-level physical memory mapping (Resident Set Size) and CPU register utilization of Node.js and a WASI container (Wasmtime) under extreme numeric and string-allocation scaling (10^8 iterations).

METHODOLOGY

The programs were executed using Termux, a free open-source terminal emulator and Linux environment app for Android that allows users to run a command-line interface on their mobile device without needing root access. The methodology employed in this work consists of three distinct stages:

1. **Environment configuration and compilation:** Install of required tools and generation of executable binaries.
2. **Deployment Instrumentation and Telemetry Capture:** Preparing the environment to measure execution metrics.
3. **Data Collection:** Gathering performance results during execution.

To do the performance benchmark between JavaScript and WebAssembly, we used four specific test files: `numeric_test.c`, `numeric_test.js`, `string_test.c`, and `string_test.js`. Each program was evaluated using three input scales, as show in Table 1.

Tabela 1 – Experimental Input Sizes

Category	Description
Small	Initial baseline testing
Medium	Standard workload simulation
Large	Stress testing and peak performance

4.1 Configuration and Compilation

The primary objective of this phase was the compilation of the source C algorithms into standalone WebAssembly architectures. While mainstream literature predominantly evaluates WebAssembly within web browser runtimes, this work specifically isolates execution behavior inside a native mobile Linux environment. To achieve this browser-independent deployment, the Clang cross-compiler was coupled with the WebAssembly System Interface (WASI). This combination allows the compiled bytecode to execute directly via systemic invocation while leveraging the LLVM backend optimization pipeline to minimize the static binary footprint ([WebAssembly Community Group, 2023](#)).

Conversely, the JavaScript test artifacts required no ahead-of-time compilation, as JavaScript relies on dynamic execution paradigms. Instead, the raw scripts were evaluated utilizing the Node.js runtime environment, which leverages the V8 engine’s Just-In-Time (JIT) compilation pipeline to translate source scripts into machine instructions dynamically at runtime (TILKOV; VINOSKI, 2010).

The orchestration of this dual-runtime environment was sustained using two core infrastructural tools: Termux and an unprivileged Debian PRoot container. Termux provided a low-overhead, native POSIX environment on the Android operating system, enabling precise interaction with the host hardware architecture and eliminating the overhead associated with heavy hypervisors (FORNWALL; CONTRIBUTORS, 2024). This environment allowed for the direct, granular capture of execution latency and process memory constraints.

To resolve cross-environment compatibility challenges and access standardized tool-chains—such as the WASI-SDK—a Debian environment was instantiated via PRoot. This architectural layout successfully bridged the gap between conventional glibc compilation demands and the specialized Bionic libc environment of the Android OS (DEGRÉMONT; CONTRIBUTORS, 2024). Together, these tools formed a stable, reproducible pipeline capable of compiling the source binaries and staging them for empirical execution.

4.2 Deployment Instrumentation and Telemetry Capture

To guarantee empirical rigor, eliminate human error, and ensure statistical reproducibility across all evaluation tiers, the execution phase was fully automated utilizing a structured POSIX shell scripting (IEEE Computer Society, 2018). Rather than executing the WebAssembly modules and JavaScript files through manual terminal interventions, the instrumentation layer orchestrated nested execution loops to sweep across all independent variables: runtime environments (Wasmtime versus Node.js), computational categories (numeric arithmetic versus string manipulation), and scaling workload (10^5 , 10^7 , and 10^8 for input sizes). Each unique configuration state was subjected to three sequential, isolated trials to provide a robust dataset for mean variance analysis. Process telemetry—specifically execution latency and peak volatile memory consumption—was captured externally to prevent the measurement apparatus from introducing internal profiling overhead or altering the execution behavior of Wasmtime or Node.js. This non-invasive abstraction was achieved by leveraging the binary tracking capabilities of the GNU time utility (/data/data/com.termux/files/usr/bin/time), executing natively within the Termux environment (Free Software Foundation, 2022). The telemetry step was configured with specific formatting expressions to isolate the required dependent variables directly from the operating system kernel:

```
/usr/bin/time -f "%e,%M" <runtime_engine> <artifact_target> <workload_size>
```


Within this instrumentation string, the `%e` formatting token instructs the kernel to measure total elapsed real time (wall-clock time) in seconds, capturing the exact duration from process instantiation to termination. Concurrently, the `%M` token queries the kernel's process tracker to log the maximum resident set size (RSS), recording the peak physical random-access memory (RAM) allocated to the process during its lifecycle, measured in kilobytes (KB) (Free Software Foundation, 2022).

To isolate these benchmarking metrics from standard program outputs, the standard output (stdout) streams of both `wasmtime` and `node` were redirected to the null device (`>/dev/null`). The telemetry command emitted by the GNU time utility via the standard error (stderr) stream was intercepted, captured as string variables within the shell environment, and dynamically appended to a persistent comma-separated values (.csv) spreadsheet payload. This architectural pipeline ensured that data acquisition remained entirely unpolluted by terminal input/output latency, securing a sterile environment for subsequent statistical analysis. To ensure reproducible research, all benchmark source code, execution scripts, and deployment configurations are publicly available in the dedicated GitHub repository: <https://github.com/MuriloMatos2004/Mobile-Wasm-Benchmark>.

4.3 Data collection

The approach utilized for data analysis was based on the arithmetic mean of the experimental trials, calculating the sample standard deviation to assess data dispersion, and computing a performance Speedup Factor (S). The Speedup Factor explicitly indicates the relative performance magnitude of one runtime environment over the other under identical workloads.

To eliminate transient background noise from the operating system and ensure the reliability of the dataset, three sequential trials were executed for each individual input workload scale. By repeating the experiments iteratively, data accuracy was significantly improved. The raw metrics from these three distinct trials were synthesized using the standard arithmetic mean equation:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (4.1)$$

where $n = 3$ represents the number of trials, and x_i represents the individual metric recorded for a specific run. Applying this equation generated a stable baseline to contrast JavaScript and WebAssembly across both execution latency (time) and memory footprint (RAM) allocation profiles.

To complement the mean and quantify the experimental stability and variance across these iterative trials, the sample standard deviation (s) was calculated for each unique testing

configuration. This statistical metric evaluates the concentration of the data points around the mean, mathematically formulated as:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (4.2)$$

A low standard deviation value confirms a high degree of precision and consistency within the isolated testing environment, verifying that individual runs were not corrupted by unexpected host OS anomalies.

Another core metric calculated was the performance ratio between the JavaScript and WebAssembly runtimes, which provides a direct, precise benchmark comparison of the two target technologies. The mathematical construction of the Speedup Factor (S) is defined as follows:

$$S = \frac{T_{\text{JavaScript}}}{T_{\text{WebAssembly}}} \quad (4.3)$$

$T_{\text{JavaScript}}$ and $T_{\text{WebAssembly}}$ represent the calculated mean execution times for each respective environment. The analytical expressions provided the necessary structured data to plot scaling trend charts and compile comparative tables, making the empirical outcomes of this research immediately interpretable.

DEVELOPMENT

5.1 The problem

The primary problem addressed is the comparative behavior of dynamic scripting environments and low-level compiled runtimes when executed on resource-constrained mobile hardware.

When executing lightweight tasks, high-level runtime engines (such as JavaScript's V8) and low-level virtual machines (such as WebAssembly's Wasmtime) often exhibit similar performance profiles. However, as the workload scale increases, each architecture encounters distinct operational constraints.

To systematically investigate this problem, it isolates how both environments manage two distinct categories of computational stress: numeric stress (heavy arithmetic loop operations) and string stress (dynamic memory allocation and text manipulation). Isolating these workloads allows for a direct evaluation of how JIT compilation overhead compares with WebAssembly's linear memory model under scaling conditions.

5.2 Realized activities

The practical execution is centered on the implementation of target test scripts across both C and JavaScript runtime environments. The core codebase comprises `numeric_test.c`, `string_test.c`, `numeric_test.js` and `string_test.js`. A critical methodological milestone involved cross-compiling the native C source files into WebAssembly bytecode binaries targeting the WebAssembly System Interface (WASI) layer to facilitate low-level native execution within the Termux sandbox environment.

To automate data acquisition, a custom POSIX shell orchestration script was deployed to iteratively invoke both the WebAssembly modules and the JavaScript code. The performance data streams are systematically organized them into a unified, persistent comma-separated values (`results.csv`) repository. Subsequently, an independent Python evaluation script was engineered to ingest this data matrix, compute statistical averages, determine the relative Speedup Factor (S), and render the final analytical charts without further relying on the shell infrastructure.

The core computational logic for the numerical and strings baseline evaluations is detailed in A. Listing 2 and 3 shows the structured native numeric and string C implementation (subsequently compiled to Wasm bytecode), while Listing 1 and 4 details the equivalent evaluation scripts constructed in JavaScript.

The architecture of the scripts highlights the systematic environments. Both implementation streams were designed to identical scales (10^5 , 10^7 , and 10^9 iterations). This volume is then fed directly into a linear mathematical calculation loop designed to enforce high, uninterrupted CPU workload saturation for execution latency tracking.

To operationalize the experimental design and eliminate variables associated with manual execution latency, the testing workflow was completely automated. This was realized through the development of a POSIX-compliant Bash script designated as `benchmark.sh`, which serves as the primary execution engine for the entire testing framework. By relying, the runtime environment maintains a strict, reproducible sequence across every evaluation cycle.

The execution of this automation script handles with three deeply nested iteration loops. These loops programmatically sweep across the chosen experiment boundaries: the 2 workload types (*numeric* and *string*), the 3 exponential input metrics (10^5 , 10^7 , and 10^9), and the 3 consecutive verification iterations designed for data reduction. This loop layout accounts for a total of 36 unique execution profiles.

During every independent loop cycle, the utility encapsulates both standard output and error descriptors (`2 > &1 > /dev/null`) to isolate the operating system kernel telemetry streams, appending the raw performance profiles directly into a persistent comma-separated database. The structural code logic of this data collection subsystem is detailed in Listing 5.

5.3 Results

5.3.1 *Numeric trials results*

The empirical telemetry gathered for the numerical evaluations reveals a clear performance divergence between the execution environments across both resource dimensions. Regarding volatile memory consumption (illustrated in Figure 1), WebAssembly (Wasmtime) consistently maintained a minimized, static memory footprint. Conversely, despite utilizing an automated Garbage Collector designed to mitigate memory, JavaScript (Node.js) exhibited a significantly larger memory footprint that scale alongside the workload intensity.

This unexpected memory inflation in the JavaScript runtime is attributed to the internal architecture of the V8 engine when handling the `BigInt` data type. To guarantee 64-bit integer arithmetic precision equivalent to C's `long long` type, JavaScript is forced to dynamically allocate memory space to handle arbitrary-precision numeric objects (V8 Project, 2018). As the stress-test loop executes millions of rapid iterative additions, a high volume of temporary

intermediate numeric allocations saturates the heap. Because the Garbage Collector operates on periodic cycles rather than an instantaneous inline basis (MARSHALL, 2019), these short-lived objects accumulate faster than the engine can reclaim them, yielding a elevated peak Resident Set Size (RSS).

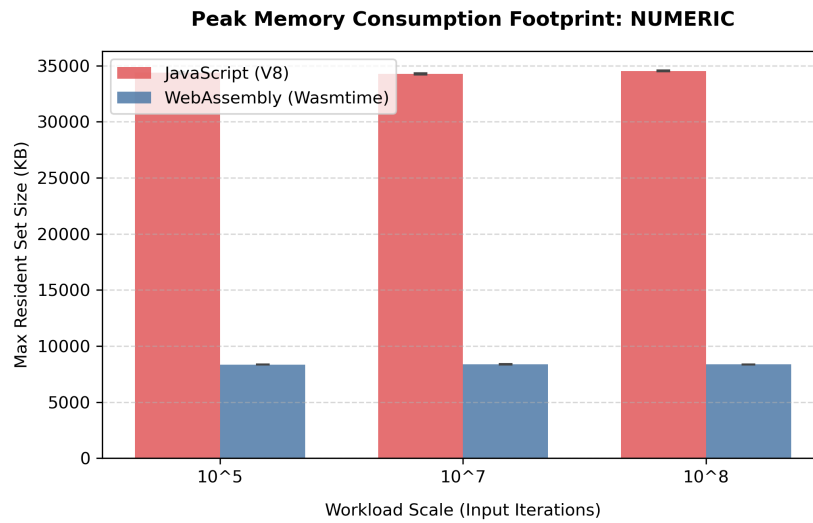


Figure 1 – Peak Memory Consumption Comparison for Numeric Workloads

WebAssembly demonstrated clear operational superiority over the JavaScript baseline. As the input parameters expanded from 10^5 to 10^8 iterations, the execution latency of JavaScript escalated, exhibiting significant runtime variations. This behavioral volatility highlights the inherent computational overhead introduced by JavaScript’s dynamic Just-In-Time (JIT) compilation layer.

This architectural difference is empirically validated by the standard deviation error bars displayed on the trend lines. For the WebAssembly execution profile, the error bars remain virtually invisible across all workload scales, confirming that the AOT bytecode runs with near-perfect deterministic latencies and zero statistical dispersion (HAAS *et al.*, 2017). Conversely, as the workload scales to 10^8 iterations, the JavaScript baseline exhibits visible error brackets. This dispersion quantifies the non-deterministic behavior of the V8 JIT compiler, which introduces minor latency fluctuations as it uses tools for optimization and for garbage collection under heavy workloads (BARRETT *et al.*, 2017).

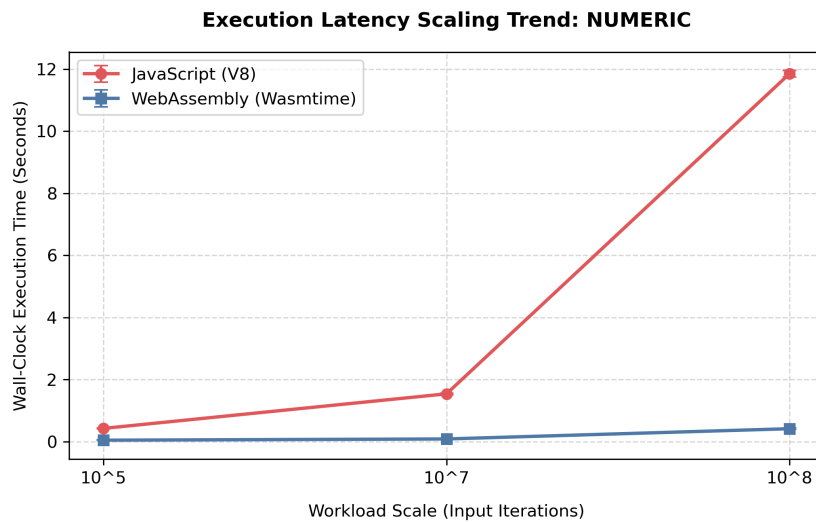


Figure 2 – Execution Latency Scaling Trend for Numeric Workloads

5.3.2 String trials results

The experimental results for the string manipulation workloads evaluate the relationship between execution throughput and runtime memory allocation. Figure 3 illustrates the peak Resident Set Size (RSS) metrics across the progressive workload scales.

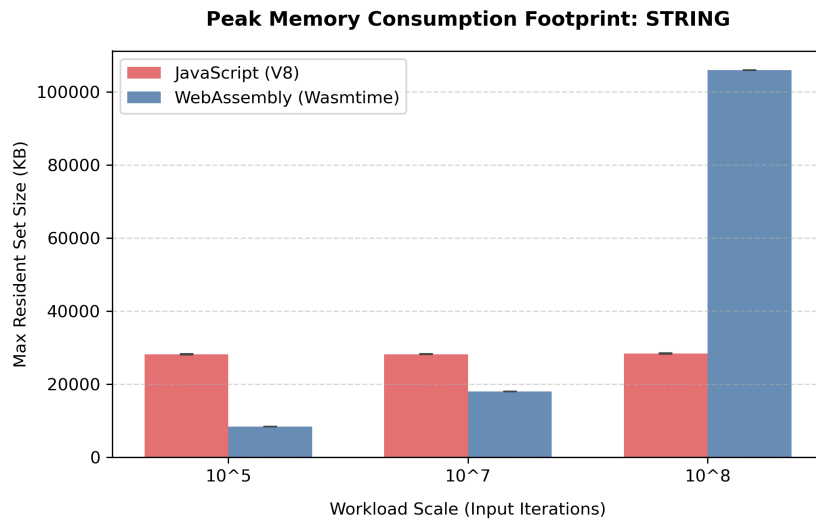


Figure 3 – Peak Memory Consumption Comparison for string Workloads

JavaScript (Node.js) demonstrated an idealized, invariant memory profile across all input scales. Conversely, WebAssembly (Wasmtime) exhibited an memory inflation at the maximum workload tier (10^8 iterations), revealing a significant vulnerability in its data processing scalability.

Under the experimental conditions, the Node.js process's peak RSS remained virtually constant across the tested loads, indicating that the V8-managed heap was sufficient to accom-

modate the observed allocations. In contrast, the WebAssembly module exhibited proportional growth in linear memory, reflecting ‘malloc’ calls made to handle the heavier workloads.. JavaScript relies on an automated Garbage Collection (GC) framework within the V8 runtime. During string manipulation operations, intermediate object allocations trigger periodic scavenging and compaction cycles, which ensures that JavaScript’s peak Resident Set Size (RSS) remains stable regardless of input scale (MARSHALL, 2019).

WebAssembly, however, operates on a sandboxed **Linear Memory** paradigm that lacks a native garbage collection layer in standalone WebAssembly System Interface (WASI) environments. When allocating dynamic character arrays, the compilation module must continuously requests additional 64 KB memory pages from the host runtime to accommodate growing string buffers. Because this memory cannot be dynamically compacted or returned to the operating system, WebAssembly experiences massive memory flow at the 10^8 threshold, validating the structural cost of unmanaged linear heaps (YAN *et al.*, 2021).

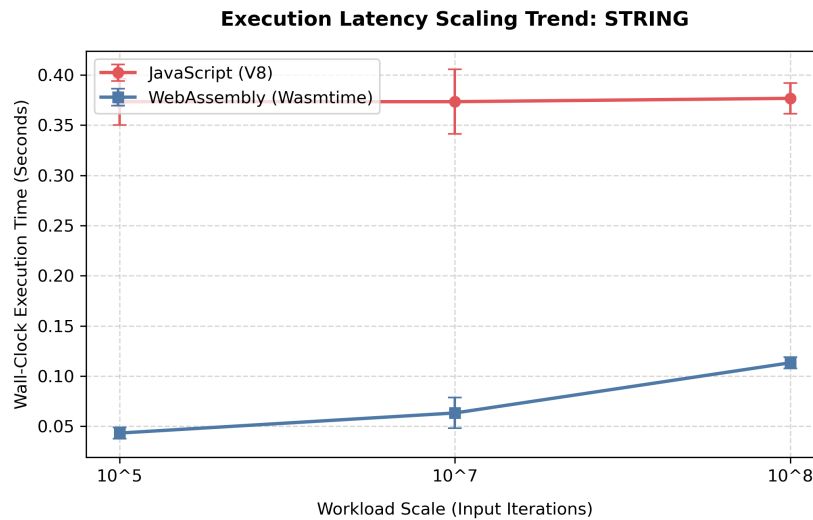


Figure 4 – Execution Latency Scaling Trend for string Workloads

The performance of JavaScript was significantly slower than WebAssembly across all evaluated scales. However, the JavaScript runtime demonstrated remarkable execution stability, scaling linearly and predictably. WebAssembly maintained its computational speed dominance but exhibited distinct behavioral fluctuations and processing volatility across the different execution runs.

This behavior is illuminated by evaluating the standard deviation error bars for each runtime. The JavaScript baseline exhibits an evenly distributed behavior with variance across the entire testing spectrum. This indicates that while the V8 engine carries a static, predictable latency penalty due to persistent garbage collection monitoring and dynamic string allocations, its execution remains uniform. Conversely, WebAssembly exhibits tightly constrained error bars that reflect a high determinism, yet its scaling curve shows that string processing incurs increasing

latency. Because WebAssembly lacks direct, native string data types within its core specification, it must manipulate strings by indexing individual bytes directly within its isolated linear memory array (YAN *et al.*, 2021). At 10^8 iterations, the compounding overhead of character-by-character boundary checking and memory offsets causes the noticeable upward slope in latency, even while maintaining low statistical dispersion.

5.3.3 *S factor table*

The empirical results compiled in Table 2 mathematically quantify the computational performance advantages of WebAssembly (Wasmtime) over the JavaScript baseline (Node.js) using the Speedup Factor (S). According to the operational definition established in Methodology, any outcome where $S > 1$ denotes a performance acceleration attributable to the low-level virtual machine architecture. The calculated metrics reveal two contrasting scaling behaviors that highlight the fundamental algorithmic efficiencies of both execution environments.

In the numeric computing domain, WebAssembly exhibits at the initial baseline of 10^5 iterations, Wasmtime achieves a substantial speedup factor of $S = 9.14$, indicating it executed the workload over nine times faster than the V8 engine. As the computational stress scales exponentially to 10^7 and 10^8 iterations, this metric accelerates to $S = 17.77$ and reaches a peak velocity delta of $S = 28.45$, respectively. This upward trajectory confirms that for pure mathematical loop structures, WebAssembly’s localized Ahead-of-Time (AOT) machine compilation scales near-linearly, bypassing the heavy monitoring, optimization, and de-optimization cycles inherent to JavaScript’s dynamic Just-In-Time (JIT) compilation layer.

Conversely, the string manipulation category demonstrates a distinct, inverted scaling trend. While WebAssembly retains its status as the absolute performance leader across all evaluated boundaries, its relative speedup multiplier contracts sharply as the workload expands. At 10^5 iterations, Wasmtime delivers an $8.62\times$ performance acceleration over JavaScript. However, as the iteration count moves to 10^7 and reaches maximum data density at 10^8 , the Speedup Factor degrades down to $S = 5.89$ and lands at its lowest recorded threshold of $S = 3.32$.

This systematic deceleration does not imply an algorithmic failure in WebAssembly’s execution pipeline, but rather exposes the severe execution latency penalties imposed by its unmanaged memory architecture. Because standalone WebAssembly relies on a strict, continuous Linear Memory layout lacking an internal Garbage Collector, the module must repeatedly pause its core execution loop to dynamically request, map, and zero-initialize new 64 KB memory pages from the host runtime to handle growing string buffers. These frequent, blocking system calls inject a progressive latency overhead that counteracts Wasm’s raw processing velocity, allowing the highly optimized, inline garbage-collected JavaScript heap to claw back relative performance at scale.

Tabela 2 – Performance Speedup Factors (S)

Computational Category	Workload Scale (Iterations)	Speedup Factor (S)	Performance Leader
Numeric Test	10^5	9.14x	Wasmtime
	10^7	17.77x	Wasmtime
	10^8	28.45x	Wasmtime
String Test	10^5	8.62x	Wasmtime
	10^7	5.89x	Wasmtime
	10^8	3.32x	Wasmtime

5.4 Difficulties and Limitations

The implementation, automated orchestration and execution of this cross-runtime evaluation pipeline surfaced several hardware-bound limitations, environment-specific system bottlenecks, and technical complexities. Documenting these constraints provides crucial context for interpreting the empirical data and establishes a technical baseline for reproducibility.

5.4.1 Methodological and Architectural Limitations

The primary hardware limitation of this research stems from the evaluation scope being confined to a single mobile processing architecture. All experiments were executed natively inside an unprivileged virtualized user space on a consumer-grade smartphone featuring a Samsung Galaxy ARM64 system-on-chip (SoC). While this choice isolates the exact behavior of mobile ARM microprocessors, the performance metrics may display variations if executed on alternative hardware configurations.

5.4.2 System Administration and Linux Environment Traps

Running the benchmark within a restricted, unprivileged Debian container revealed software dependency gaps. The automation script, `benchmark.sh`, initially failed because it relied on the standalone system utility located at `/usr/bin/time` to format performance metrics using the `-f "%e,%M"` flag.

Because this specific binary was missing from the container, the system fell back to the default internal Bash keyword implementation of `time`. Since the built-in shell version lacks advanced formatting flags, it threw critical execution errors that corrupted the early data collections. This issue required a manual re-configuration to explicitly install the standalone GNU `time` package into the sandboxed environment.

5.4.3 Runtime Memory Constraints

When executing the JavaScript iteration loop on Node.js at the 10^9 data density mark, the engine abruptly crashed with a fatal `RangeError: Invalid string length`. In V8, strings

are structurally bounded to a maximum length of 512 Megabytes on 32-bit platforms and approximately 1 Gigabyte on 64-bit systems. Because the iterative string concatenation algorithm attempted to continuously double or linearize memory buffers beyond this hardcoded constraint, the engine threw a critical memory error, forcing a defensive reduction of the maximum string experiment array bounds to maintain systemic runtime integrity.

CONCLUSION

The work provides academic contributions by empirically quantifying the Speedup Factors and resource trade-offs between managed scripting runtimes and low-level bytecode virtual machines executing on mobile hardware. By delivering a POSIX-compliant automated tailored to sandboxed Android and Debian, it was established the reproducibility. Additionally, the empirical evidence shows that selecting an execution layer for mobile systems is not a binary decision based entirely on execution speed, but rather a calculated structural trade-off between runtime execution stability and physical memory footprints.

While it successfully was isolated the core performance profiles of both technologies, several paths remain open for future investigation. First, upcoming studies should evaluate the newly introduced WebAssembly Garbage Collection (WasmGC) extension. This will determine if adding a native managed heap to WebAssembly can eliminate the severe memory use found during heavy string operations without slowing down its processing speed.

Secondly, the benchmark can be expanded from simple single-threaded loops to parallel processing tasks. Utilizing technologies such as Web Workers for JavaScript and WebAssembly Threads will demonstrate how effectively both environments distribute workloads across the multiple CPU cores found in modern smartphone processors. Furthermore, the code used in the experiments could be refined to provide a more representative comparison between JavaScript and WebAssembly. Instead of using distinct data structures—such as strings in JavaScript and memory buffers in C/WebAssembly—future versions of the benchmark could employ equivalent structures and identical operations in both environments. This refinement would minimize the influence of implementation differences between the languages and runtimes, allowing the results to more accurately reflect the impact of each technology’s memory management model.

Finally, porting this automated test to other hardware architectures would help validate the results. Testing the same software on desktop x86-64 processors or low-power embedded RISC-V boards will reveal if the performance crossover points and resource limits discovered in this thesis apply universally across different types of computers and devices.

CITATIONS AND REFERENCES

REFERENCES

BARRETT, E.; BOLZ-TEREICK, C. F.; KILLICK, L.; TRATT, L. Virtual machine warm-up blows hot and cold. **Proceedings of the ACM on Programming Languages**, ACM, v. 1, n. OOPSLA, p. 1–27, 2017. Citado 4 vezes nas páginas 23, 24, 27 e 35.

DEGRÉMONT, C.; CONTRIBUTORS. **PRoot: User-space chroot, mount, and binfmt_misc**. 2024. Accessed: June 2026. Disponível em: <<https://proot-me.github.io>>. Citado na página 30.

ENGELKE, A.; SCHWARZ, T. Compile-time analysis of compiler frameworks for query compilation. In: **2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)**. [S.l.: s.n.], 2024. p. 233–244. Citado na página 24.

FLANAGAN, D. **JavaScript: The Definitive Guide**. [S.l.]: O'Reilly Media, Inc., 2020. Citado na página 21.

FORNWALL, F.; CONTRIBUTORS. **Termux: A Linux Environment Android Application**. 2024. Accessed: June 2026. Disponível em: <<https://github.com/termux/termux-app>>. Citado na página 30.

Free Software Foundation. **GNU time: Resource Usage Measurement Utility**. [S.l.], 2022. GNU Version 1.9 Manual. Disponível em: <<https://www.gnu.org/software/time/>>. Citado 2 vezes nas páginas 30 e 31.

HAAS, A.; ROSSBERG, A.; SCHUFF, D. L.; TITZER, B. L.; GOHMAN, D.; WAGNER, L.; ZAKAI, A.; KINNUNEN, J.; HOLMAN, L. Bringing the web up to speed with WebAssembly. In: **Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation**. [S.l.: s.n.], 2017. p. 185–200. Citado na página 35.

HOCKLEY, D.; WILLIAMSON, C. Benchmarking runtime scripting performance in wasmer. In: **Companion of the 2022 ACM/SPEC International Conference on Performance Engineering**. [S.l.: s.n.], 2022. p. 97–104. Nenhuma citação no texto.

IEEE Computer Society. **The Open Group Base Specifications Issue 7 (POSIX.1-2017)**. [S.l.]: IEEE Std 1003.1-2017, 2018. (Standard for Information Technology). Citado na página 30.

MARICQ, J.; DUPLYAKIN, D.; JIMENEZ, R.; RICCI, R.; STUTSMAN, R. An evaluation of memory telemetry in virtualized and containerized environments. In: **Proceedings of the 2018 ACM/SPEC International Conference on Performance Engineering**. [S.l.: s.n.], 2018. p. 105–116. Citado na página 26.

MARSHALL, P. **Trash talk: the Orinoco garbage collector**. 2019. <<https://v8.dev/blog/trash-talk>>. V8 JavaScript Engine Architecture Journal. Accessed: 2026-06-15. Citado 2 vezes nas páginas 35 e 37.

RAMESH, A.; HUANG, T.; TITZER, B. L.; ROWE, A. Empowering webassembly with thin kernel interfaces. In: **Proceedings of the European Conference on Computer Systems (EuroSys)**. [S.l.: s.n.], 2023. Citado 2 vezes nas páginas 24 e 27.

RICHARDS, G.; LEBRESNE, S.; BURG, B.; VITEK, J. An analysis of the dynamic behavior of javascript programs. **ACM SIGPLAN Notices**, ACM, v. 45, n. 6, p. 1–12, 2010. Citado 2 vezes nas páginas 23 e 27.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10th. ed. [S.l.]: John Wiley & Sons, 2018. Citado na página 25.

Statcounter Global Stats. **Mobile Operating System Market Share Worldwide**. 2026. Accessed: June 2026. Disponível em: <<https://gs.statcounter.com/os-market-share/mobile/worldwide>>. Citado na página 21.

TANENBAUM, A. S.; BOS, H. **Modern Operating Systems**. 4th. ed. [S.l.]: Pearson, 2015. Citado na página 25.

TILKOV, S.; VINOSKI, S. Node.js: Using JavaScript to build high-performance network programs. **IEEE Internet Computing**, IEEE, v. 14, n. 6, p. 80–83, 2010. Citado na página 30.

V8 Project. **Introducing BigInt in JavaScript**. 2018. <<https://v8.dev/features/bigint>>. Accessed: 2026-06-15. Citado na página 34.

WebAssembly Community Group. **WebAssembly System Interface (WASI) Core Specification**. [S.l.], 2023. Disponível em: <<https://wasi.dev>>. Citado na página 29.

YAN, Y.; TU, T.; ZHAO, L.; ZHOU, Y. G.; WANG, W. Understanding the performance of WebAssembly applications. In: **Proceedings of the 2021 ACM Internet Measurement Conference (IMC)**. [S.l.]: ACM, 2021. p. 533–549. Citado 4 vezes nas páginas 21, 28, 37 e 38.

ZHANG, Y.; LIU, M.; WANG, H.; MA, Y.; HUANG, G.; LIU, X. Research on webassembly runtimes: A survey. **arXiv preprint arXiv:2404.12621**, 2024. Citado 2 vezes nas páginas 24 e 27.

Appendix ASCRIPTS

Source code 1: Equivalent Numerical Benchmark in JavaScript (numeric_test.js)

```
1 // 1. Argument Parsing (Equivalent to argc checking)
2 // process.argv[0] is the node binary path
3 // process.argv[1] is the script file path (numeric_test.js)
4 // process.argv[2] is the first user argument (the number)
5 if (process.argv.length < 3) {
6     console.log("Usage: node numeric_test.js <number>");
7     process.exit(1);
8 }
9
10 // 2. Data Type Transformation (Equivalent to atoll)
11 // BigInt is required here because JavaScript standard numbers
12 // lose precision above 9 quadrillion (the "Safe Integer" limit).
13 // BigInt forces 64-bit integer math, matching C's 'long long'.
14 const inputSize =
15 BigInt(process.argv[2]);
16 let sum = 0n; // The 'n' suffix denotes a BigInt literal in
17     JavaScript
18
19 // 3. The Computational Core (The Stress Test Loop)
20 for (let i = 0n; i < inputSize; i++) {
21     sum += i;
22 }
23
24 // 4. Output and Exit
25 console.log('Result: ${sum.toString()}');
26 process.exit(0);
```

Source code 2: Numerical Benchmark Implementation in C (numeric_test.c)

```
1 #include <stdio.h>
2 #include <stdlib.h> // Needed for atoll (string to long long)
3
4 int main(int argc, char *argv[]) {
```

```
5 // This takes the number from the command line
6 if (argc < 2) {
7     printf("Usage: ./test.wasm <number>\n");
8     return 1;
9 }
10
11 long long input_size = atoll(argv[1]);
12 long long sum = 0;
13
14 for (long long i = 0; i < input_size; i++) {
15     sum += i;
16 }
17
18 printf("Result: %lld\n", sum);
19 return 0;
20 }
```

Source code 3: string Benchmark Implementation in C (string_test.c)

```
1
2 int main(int argc, char *argv[]) {
3     if (argc < 2) {
4         printf("Usage: %s <size>\n", argv[0]);
5         return 1;
6     }
7
8     long long size = atoll(argv[1]);
9
10    // Dynamically allocate memory based on the terminal argument
11    char *str = (char *)malloc(size + 1);
12    if (str == NULL) {
13        printf("Memory allocation failed!\n");
14        return 1;
15    }
16
17    // Fill the memory to simulate data processing
18    memset(str, 'A', size);
19    str[size] = '\0'; // Null-terminator
20
21    printf("Result: Processed memory buffer of size %lld bytes\n",
22           size);
23
24    free(str); // Clean up memory
```

```

24     return 0;
25 }

```

Source code 4: string Benchmark Implementation in JS (string_test.js)

```

1
2 // string_test.js
3 const inputSize = parseInt(process.argv[2]);
4
5 if (isNaN(inputSize)) {
6     console.log("Usage: node string_test.js <number>");
7     process.exit(1);
8 }
9
10 // Dynamically allocate space and repeat a string pattern
11 // If inputSize is 100000, it creates a string of 100,000 'A's
12 let testString = "A".repeat(inputSize);
13
14 // Perform an operation on it to force the CPU/RAM to work
15 let stringLength = testString.length;
16
17 console.log(`Result: Processed string of length ${stringLength}`)
18 ;

```

Source code 5: Automated Data Collection Orchestration Harness (benchmark_sh)

```

1 #!/bin/bash
2
3 # 1. Establish the output spreadsheet destination
4 OUTPUT_FILE="results.csv"
5
6 # 2. Write the CSV headers (Crucial so you know what the columns
7      mean)
8 echo "Language,Category,Input_Size,Run,Time_Seconds,Max_RAM_KB" >
9      $OUTPUT_FILE
10
11 # 3. Define the complete experimental parameters
12 CATEGORIES=("numeric" "string")
13 SIZES=("100000" "10000000" "100000000")
14 REPEATS=(1 2 3)
15
16 echo "Starting Thesis Benchmark Pipeline..."
17 echo "-----"

```

```

16
17 for cat in "${CATEGORIES[@]}"; do
18     for size in "${SIZES[@]}"; do
19         for run in "${REPEATS[@]}"; do
20
21             # --- PROCESS 1: WEBASSEMBLY EXECUTION & CAPTURE ---
22             echo "Running WASM | Category: $cat | Size: $size |
Run: $run"
23
24             # Execute, isolate metrics, and store in a variable
25             WASM_METRICS=$(/usr/bin/time -f "%e,%M" wasmtime "${
cat}_test.wasm" $size 2>&1 >/dev/null)
26
27             # SAVE TO CSV: Append the data to your file
28             echo "Wasm,$cat,$size,$run,$WASM_METRICS" >>
$OUTPUT_FILE
29
30
31             # --- PROCESS 2: JAVASCRIPT EXECUTION & CAPTURE ---
32             echo "Running JS | Category: $cat | Size: $size |
Run: $run"
33
34             # Execute, isolate metrics, and store in a variable
35             JS_METRICS=$(/usr/bin/time -f "%e,%M" node "${cat}
_test.js" $size 2>&1 >/dev/null)
36
37             # SAVE TO CSV: Append the data to your file
38             echo "JavaScript,$cat,$size,$run,$JS_METRICS" >>
$OUTPUT_FILE
39
40             done
41         done
42     done
43
44     echo "-----"
45     echo "Benchmark Complete! 36 runs processed. Data secured in
$OUTPUT_FILE"

```

Source code 6: python script chart

```

1 #!/usr/bin/env python3
2 import csv
3 from collections import defaultdict

```

```

4 import matplotlib.pyplot as plt
5 import statistics # Added to compute standard deviation
6
7 INPUT_FILE = "results.csv"
8
9 def generate_charts():
10     # Gather data points
11     time_data = defaultdict(lambda: defaultdict(lambda:
12         defaultdict(list)))
13     ram_data = defaultdict(lambda: defaultdict(lambda:
14         defaultdict(list)))
15
16     try:
17         with open(INPUT_FILE, mode='r', newline='', encoding='utf
18             -8') as f:
19             reader = csv.DictReader(f)
20             for row in reader:
21                 cat = row['Category']
22                 size = int(row['Input_Size'])
23                 lang = row['Language']
24                 try:
25                     time_data[cat][size][lang].append(float(row['
26                         Time_Seconds']))
27                     ram_data[cat][size][lang].append(float(row['
28                         Max_RAM_KB']))
29                 except ValueError:
30                     continue # Gracefully skip any string
31                     allocation crashes
32             except FileNotFoundError:
33                 print(f"Error: {INPUT_FILE} not found.")
34                 return
35
36     # Process each category (numeric, string, etc.) separately
37     for cat in time_data.keys():
38         sizes = sorted(time_data[cat].keys())
39
40         # --- CALCULATE MEANS AND STANDARD DEVIATIONS ---
41         js_times_mean = []
42         js_times_std = []
43         wasm_times_mean = []
44         wasm_times_std = []

```

```

40     js_rams_mean = []
41     js_rams_std = []
42     wasm_rams_mean = []
43     wasm_rams_std = []
44
45     for s in sizes:
46         # JavaScript Time Stats
47         js_t_list = time_data[cat][s]['JavaScript']
48         js_times_mean.append(statistics.mean(js_t_list) if
js_t_list else 0)
49         js_times_std.append(statistics.stdev(js_t_list) if
len(js_t_list) > 1 else 0)
50
51         # Wasm Time Stats
52         wasm_t_list = time_data[cat][s]['Wasm']
53         wasm_times_mean.append(statistics.mean(wasm_t_list)
if wasm_t_list else 0)
54         wasm_times_std.append(statistics.stdev(wasm_t_list)
if len(wasm_t_list) > 1 else 0)
55
56         # JavaScript RAM Stats
57         js_r_list = ram_data[cat][s]['JavaScript']
58         js_rams_mean.append(statistics.mean(js_r_list) if
js_r_list else 0)
59         js_rams_std.append(statistics.stdev(js_r_list) if len
(js_r_list) > 1 else 0)
60
61         # Wasm RAM Stats
62         wasm_r_list = ram_data[cat][s]['Wasm']
63         wasm_rams_mean.append(statistics.mean(wasm_r_list) if
wasm_r_list else 0)
64         wasm_rams_std.append(statistics.stdev(wasm_r_list) if
len(wasm_r_list) > 1 else 0)
65
66         print(f"\n--- Speedup Factor Analysis for {cat.upper()}
---")
67         for i, s in enumerate(sizes):
68             j_t = js_times_mean[i]
69             w_t = wasm_times_mean[i]
70             if w_t > 0 and j_t > 0:
71                 speedup_S = j_t / w_t
72                 print(f"Scale 10^{len(str(s))-1}: WebAssembly is

```

```

{speedup_S:.2f}x faster than JS (S = {speedup_S:.2f})"
73         else:
74             print(f"Scale 10^{len(str(s))-1}: Speedup
calculation omitted (missing data or crash)")
75         print("-" * 45)
76
77         # Convert sizes to string labels for clean chart axis
steps
78         size_labels = [f"10^{len(str(s))-1}" for s in sizes]
79
80         # --- CHART 1: EXECUTION LATENCY LINE GRAPH WITH ERROR
BARS ---
81         plt.figure(figsize=(7, 4.5))
82
83         # Using errorbar instead of plot to display standard
deviation
84         plt.errorbar(size_labels, js_times_mean, yerr=
js_times_std, marker='o', linewidth=2,
85                       color='#E15759', label='JavaScript (V8)',
capsize=4, elinewidth=1.5)
86         plt.errorbar(size_labels, wasm_times_mean, yerr=
wasm_times_std, marker='s', linewidth=2,
87                       color='#4E79A7', label='WebAssembly (
Wasmtime)', capsize=4, elinewidth=1.5)
88
89         plt.title(f'Execution Latency Scaling Trend: {cat.upper()
}', fontsize=12, fontweight='bold', pad=15)
90         plt.xlabel('Workload Scale (Input Iterations)', fontsize
=10, labelpad=10)
91         plt.ylabel('Wall-Clock Execution Time (Seconds)',
fontsize=10)
92         plt.grid(True, linestyle='--', alpha=0.5)
93         plt.legend(loc='upper left')
94         plt.tight_layout()
95
96         time_chart_name = f"chart_{cat.lower()}_time.png"
97         plt.savefig(time_chart_name, dpi=300)
98         plt.close()
99         print(f"Saved execution time chart: {time_chart_name}")
100
101         # --- CHART 2: MAXIMUM RAM CONSUMPTION FOOTPRINT WITH
ERROR BARS ---

```

```

102     plt.figure(figsize=(7, 4.5))
103     x = range(len(sizes))
104     width = 0.35
105
106     # Passing standard deviation directly into the yerr
parameter of the bar charts
107     plt.bar([i - width/2 for i in x], js_rams_mean, width,
yerr=js_rams_std,
108             label='JavaScript (V8)', color='#E15759', alpha
=0.85, capsize=4, ecolor='#444444')
109     plt.bar([i + width/2 for i in x], wasm_rams_mean, width,
yerr=wasm_rams_std,
110             label='WebAssembly (Wasmtime)', color='#4E79A7',
alpha=0.85, capsize=4, ecolor='#444444')
111
112     plt.title(f'Peak Memory Consumption Footprint: {cat.upper
113     (')}', fontsize=12, fontweight='bold', pad=15)
114     plt.xlabel('Workload Scale (Input Iterations)', fontsize
=10, labelpad=10)
115     plt.ylabel('Max Resident Set Size (KB)', fontsize=10)
116     plt.xticks(x, size_labels)
117     plt.grid(axis='y', linestyle='--', alpha=0.5)
118     plt.legend(loc='upper left')
119     plt.tight_layout()
120
121     ram_chart_name = f"chart_{cat.lower()}_ram.png"
122     plt.savefig(ram_chart_name, dpi=300)
123     plt.close()
124     print(f"Saved memory allocation chart: {ram_chart_name}")
125
126 if __name__ == "__main__":
    generate_charts()

```